

WDR ソケット通信
サンプルプログラム
(Windows C#)

内容

WDR ソケット通信 サンプルプログラム (Windows C#)	1
1. 概要	5
1.1. システム概要	5
2. 開発環境	5
3. アプリケーション概要	6
3.1. コマンド操作説明	6
3.1.1. コマンド一覧	6
3.1.2. 送信機状態変化通知コマンド	7
3.1.3. カウント値通知コマンド	7
3.1.4. 信号灯表示変化通知コマンド	7
3.1.5. 送信機状態取得 要求・応答コマンド	7
3.1.6. 送信機一覧取得 要求・応答コマンド	7
3.1.7. 送信機情報取得 要求・応答コマンド	7
3.1.8. 送信機呼び出し表示 要求・応答コマンド	7
3.1.9. シリアルデータ出力 要求・応答コマンド	8
3.1.10. 信号灯表示制御 要求・応答コマンド	8
3.1.11. 信号灯表示解除 要求・応答コマンド	8
3.1.12. カウント値登録 要求・応答コマンド	8
3.1.13. 受信機情報取得 要求・応答コマンド	9
3.1.14. 受信機リセット 要求・応答コマンド	9
3.1.15. カウント値取得 要求・応答コマンド	9
3.1.16. 信号灯表示取得 要求・応答コマンド	9
3.2. 関数説明	10
3.2.1. 関数一覧	10
3.2.2. WDR に接続する	11
3.2.3. ソケットをクローズ	11
3.2.4. 受信したデータをコマンドごとに分割する	12
3.2.5. 要求コマンドを送信し応答コマンドを受信する	12
3.2.6. 通知コマンドを受信する	13
3.2.7. 送信機状態変化通知を受信する	14
3.2.8. カウント値通知を受信する	14
3.2.9. 信号灯表示変化通知を受信する	15
3.2.10. 送信機状態取得要求を送信し、送信機状態取得応答を受信する	16
3.2.11. 送信機一覧取得要求を送信し、送信機一覧取得応答を受信する	16
3.2.12. 送信機情報取得要求を送信し、送信機情報取得応答を受信する	17

3.2.13.	送信機呼び出し表示要求を送信し、送信機呼び出し表示応答を受信する	18
3.2.14.	シリアルデータ出力要求を送信し、シリアルデータ出力応答を受信する.....	18
3.2.15.	信号灯表示制御要求を送信し、信号灯表示制御応答を受信する	19
3.2.16.	信号灯表示解除要求を送信し、信号灯表示解除応答を受信する	20
3.2.17.	カウント値登録要求を送信し、カウント値登録応答を受信する.....	21
3.2.18.	受信機情報取得要求を送信し、受信機情報取得応答を受信する	21
3.2.19.	受信機リセット要求を送信し、受信機リセット応答を受信する	22
3.2.20.	カウント値取得要求を送信し、カウント値取得応答を受信する.....	23
3.2.21.	信号灯表示取得要求を送信し、信号灯表示取得応答を受信する	23
3.3.	定数説明	25
3.3.1.	受信タイムアウトコード.....	25
3.3.2.	製品区分	25
3.3.3.	WDR 識別子	25
3.3.4.	拡張	25
3.3.5.	コマンド種別	25
3.3.6.	コマンドモード.....	25
3.3.7.	コマンドモード別受信タイムアウト時間.....	26
3.3.8.	PNS コマンドの応答データ	26
3.4.	構造体説明.....	27
3.4.1.	バージョン構造体	27
3.4.2.	WDT バージョン情報構造体	27
3.4.3.	WDT 情報構造体	27
3.4.4.	ベースユニット情報構造体	27
3.4.5.	WDT 状態情報構造体	28
3.4.6.	RS232C データ情報構造体	28
3.4.7.	送信機状態変化通知構造体.....	28
3.4.8.	カウント値通知構造体	29
3.4.9.	信号灯表示変化通知構造体.....	30
3.4.10.	送信機状態取得構造体.....	30
3.4.11.	送信機一覧取得構造体.....	31
3.4.12.	送信機情報取得構造体.....	31
3.4.13.	送信機情報取得拡張構造体.....	32
3.4.14.	送信機呼び出し表示構造体.....	33
3.4.15.	シリアルデータ出力要求構造体	33
3.4.16.	シリアルデータ出力応答構造体	33
3.4.17.	信号灯表示制御要求構造体.....	33
3.4.18.	信号灯表示制御応答構造体.....	34
3.4.19.	信号灯表示解除構造体.....	34

3.4.20.	カウント値登録要求構造体	35
3.4.21.	カウント値登録応答構造体	35
3.4.22.	受信機情報取得構造体	35
3.4.23.	受信機リセット構造体	36
3.4.24.	カウント値取得構造体	36
3.4.25.	信号灯表示取得構造体	36
4.	プログラム概要	38
4.1.	WDRに接続	38
4.2.	ソケットをクローズ	39
4.3.	受信したデータをコマンドごとに分割する	40
4.4.	要求コマンドを送信し応答コマンドを受信する	41
4.5.	通知コマンドを受信する	44
4.6.	送信機状態変化通知を受信する	46
4.7.	カウント値通知を受信する	48
4.8.	信号灯表示変化通知を受信する	50
4.9.	送信機状態取得要求を送信し、送信機状態取得応答を受信する	52
4.10.	送信機一覧取得要求を送信し、送信機一覧取得応答を受信する	55
4.11.	送信機情報取得要求を送信し、送信機情報取得応答を受信する	57
4.12.	送信機呼び出し表示要求を送信し、送信機呼び出し表示応答を受信する	60
4.13.	シリアルデータ出力要求を送信し、シリアルデータ出力応答を受信する	62
4.14.	信号灯表示制御要求を送信し、信号灯表示制御応答を受信する	64
4.15.	信号灯表示解除要求を送信し、信号灯表示解除応答を受信する	66
4.16.	カウント値登録要求を送信し、カウント値登録応答を受信する	68
4.17.	受信機情報取得要求を送信し、受信機情報取得応答を受信する	70
4.18.	受信機リセット要求を送信し、受信機リセット応答を受信する	72
4.19.	カウント値取得要求を送信し、カウント値取得応答を受信する	74
4.20.	信号灯表示取得要求を送信し、信号灯表示取得応答を受信する	77

1. 概要

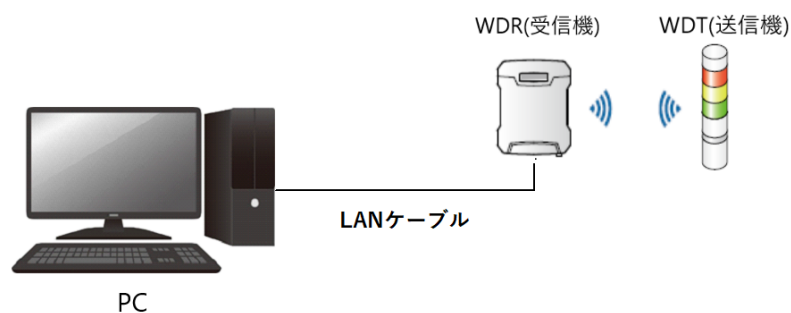
WDR をソケット通信で制御するための、サンプルプログラムの概要を記載する。

本プログラムは、パトライトが提供する DLL を使用せずに Microsoft Visual C#での制御をおこなうことを目的としている。

1.1. システム概要

本プログラムのシステム構成図は以下の通り。

本プログラムでは、1 台の WDR の機器をソケット通信で制御を行う。



2. 開発環境

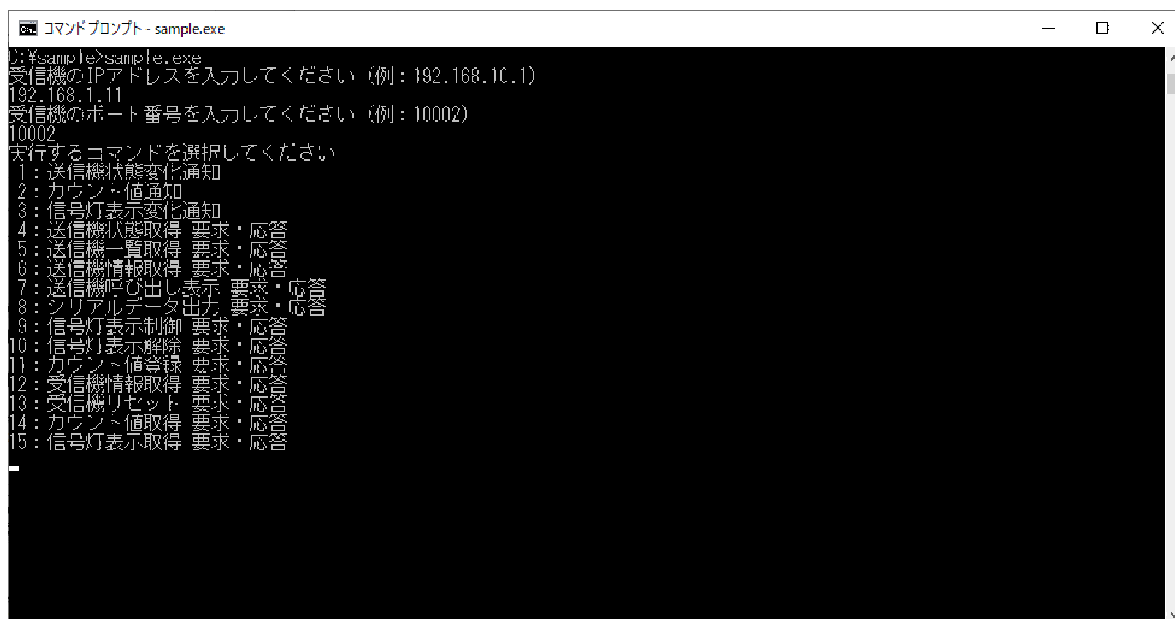
サンプルプログラムの開発環境を以下に示す。

開発環境		備考
開発 OS	Windows10 64bit	
開発言語	C#	.Net Framework 4.5 以降
アプリ種別	CUI アプリケーション	
開発ツール	VisualStudio2019 Professional	

3. アプリケーション概要

3.1. コマンド操作説明

コマンドプロンプト上で、ビルド時に作成した sample.exe のファイルの場所に移動し実行します。受信機の IP アドレスとポート番号を入力すると実行するコマンドを選択する画面になるので、番号を入力してコマンドを選択します。コマンド選択後、パラメータ入力を指示された場合は、指示に従って入力します。



3.1.1. コマンド一覧

コマンド名	内容
送信機状態変化通知	送信機の状態が変化した場合に通知されるコマンドを受信します。
カウンタ値通知	送信機のカウンタ値更新時に通知されるコマンドを受信します。
信号灯表示変化通知	送信機の表示制御状態が解除された時に通知されるコマンドを受信します。
送信機状態取得 要求・応答	送信機の状態情報(変化情報)を取得します。
送信機一覧取得 要求・応答	受信機が管理している送信機一覧を取得します。
送信機情報取得 要求・応答	指定した送信機の送信機情報を取得します。
送信機呼び出し表示 要求・応答	指定した送信機に対して呼び出し表示をします。
シリアルデータ出力 要求・応答	指定した送信機の RS232C インターフェースからシリアルデータを出力します。
信号灯表示制御 要求・応答	指定した送信機の信号灯表示を制御します。
信号灯表示解除 要求・応答	指定した送信機の信号灯表示制御を解除します。
カウンタ値登録 要求・応答	指定した送信機のカウンタ値を登録します。
受信機情報取得 要求・応答	受信機の情報を取得します。
受信機リセット 要求・応答	受信機のリセットを実施します。
カウンタ値取得 要求・応答	指定した送信機のカウンタ値を取得します。
信号灯表示取得 要求・応答	指定した送信機の信号灯表示状態を取得します。

3.1.2. 送信機状態変化通知コマンド

以下のコマンド ID を指定して実行する。

No.	入力値	値
1	コマンド ID	1

3.1.3. カウント値通知コマンド

以下のコマンド ID を指定して実行する。

No.	入力値	値
1	コマンド ID	2

3.1.4. 信号灯表示変化通知コマンド

以下のコマンド ID を指定して実行する。

No.	入力値	値
1	コマンド ID	3

3.1.5. 送信機状態取得 要求・応答コマンド

以下のコマンド ID とパラメータを指定して実行する。

No.	入力値	値
1	コマンド ID	4
2	送信機の IEEE アドレス	状態取得する送信機の IEEE アドレス

3.1.6. 送信機一覧取得 要求・応答コマンド

以下のコマンド ID を指定して実行する。

No.	入力値	値
1	コマンド ID	5

3.1.7. 送信機情報取得 要求・応答コマンド

以下のコマンド ID とパラメータを指定して実行する。

No.	入力値	値
1	コマンド ID	6
2	送信機の IEEE アドレス	情報取得する送信機の IEEE アドレス

3.1.8. 送信機呼び出し表示 要求・応答コマンド

以下のコマンド ID とパラメータを指定して実行する。

No.	入力値	値
1	コマンド ID	7
2	送信機の IEEE アドレス	呼び出し表示する送信機の IEEE アドレス

3.1.9. シリアルデータ出力 要求・応答コマンド

以下のコマンド ID とパラメータを指定して実行する。

No.	入力値	値
1	コマンド ID	8
2	送信機の IEEE アドレス	シリアルデータを送信機から出力する送信機の IEEE アドレス
3	通し番号	再送信を判断するための番号
4	出力情報	出力するデータ

3.1.10. 信号灯表示制御 要求・応答コマンド

以下のコマンド ID とパラメータを指定して実行する。

No.	入力値	値
1	コマンド ID	9
2	送信機の IEEE アドレス	信号灯の表示を制御する送信機の IEEE アドレス
3	制御時間	信号灯の表示を制御する時間
4	赤ユニットの点灯パターン	LED ユニットの制御方法 00: 制御線による制御 10: 消灯 11: 点灯 12: 点滅 13: トリプルフラッシュ
5	黄ユニットの点灯パターン	
6	緑ユニットの点灯パターン	
7	青ユニットの点灯パターン	
8	白ユニットの点灯パターン	
9	ブザーパターン	ブザーユニットの制御方法 00: 制御線による制御 10: 非吹鳴 11: 吹鳴 12: 断続吹鳴

3.1.11. 信号灯表示解除 要求・応答コマンド

以下のコマンド ID とパラメータを指定して実行する。

No.	入力値	値
1	コマンド ID	10
2	送信機の IEEE アドレス	信号灯の表示制御を解除する送信機の IEEE アドレス

3.1.12. カウント値登録 要求・応答コマンド

以下のコマンド ID とパラメータを指定して実行する。

No.	入力値	値
1	コマンド ID	11
2	送信機の IEEE アドレス	カウント値を設定する送信機の IEEE アドレス
3	カウント登録値	登録するカウント値

3.1.13. 受信機情報取得 要求・応答コマンド

以下のコマンド ID を指定して実行する。

No.	入力値	値
1	コマンド ID	12

3.1.14. 受信機リセット 要求・応答コマンド

以下のコマンド ID を指定して実行する。

No.	入力値	値
1	コマンド ID	13

3.1.15. カウント値取得 要求・応答コマンド

以下のコマンド ID とパラメータを指定して実行する。

No.	入力値	値
1	コマンド ID	14
2	送信機の IEEE アドレス	カウント値を取得する送信機の IEEE アドレス

3.1.16. 信号灯表示取得 要求・応答コマンド

以下のコマンド ID とパラメータを指定して実行する。

No.	入力値	値
1	コマンド ID	15
2	送信機の IEEE アドレス	信号灯の制御状態を取得する送信機の IEEE アドレス

3.2. 関数説明

3.2.1. 関数一覧

関数名	説明
SocketOpen	WDRに接続する
SocketClose	ソケットをクローズする
RecvDataguide	受信したデータをコマンドごとに分割する
SendCommand	要求コマンドを送信し応答コマンドを受信する
RecvCommand	通知コマンドを受信する
WDR_StatusChangeNoticeCommand	送信機状態変化通知を受信する
WDR_CountNoticeCommand	カウント値通知を受信する
WDR_SignalLightChangeNoticeCommand	信号灯表示変化通知を受信する
WDR_TransmitterStatusRequest	送信機状態取得要求を送信し、送信機状態取得応答を受信する
WDR_TransmitterListRequest	送信機一覧取得要求を送信し、送信機一覧取得応答を受信する
WDR_TransmitterDataRequest	送信機情報取得要求を送信し、送信機情報取得応答を受信する
WDR_TransmitterCallRequest	送信機呼び出し表示要求を送信し、送信機呼び出し表示応答を受信する
WDR_SerialOutputRequest	シリアルデータ出力要求を送信し、シリアルデータ出力応答を受信する
WDR_SignalLightControlRequest	信号灯表示制御要求を送信し、信号灯表示制御応答を受信する
WDR_SignalLightLiftRequest	信号灯表示解除要求を送信し、信号灯表示解除応答を受信する
WDR_SignalLightCountSetRequest	カウント値登録要求を送信し、カウント値登録応答を受信する
WDR_ReceiveDataRequest	受信機情報取得要求を送信し、受信機情報取得応答を受信する
WDR_ReceiverResetRequest	受信機リセット要求を送信し、受信機リセット応答を受信する
WDR_SignalLightCountGetRequest	カウント値取得要求を送信し、カウント値取得応答を受信する
WDR_SignalLightDataGetRequest	信号灯表示取得要求を送信し、信号灯表示取得応答を受信する

3.2.2. WDRに接続する

関数名	public static int SocketOpen(string ip, int port)	
パラメータ	string ip	WDR の IP アドレス
	int port	WDR のポート番号
戻り値	int	成功:0、失敗:0 以外
説明	指定した IP アドレスとポート番号の WDR にソケット通信で接続する	
関数の使用方法	<pre>// Socket クラスの変数を定義 private static Socket sock = null; // メイン関数 static void Main() { // WDR に接続 ret = SocketOpen("192.168.10.1", 10002); if (ret == -1) { return; } }</pre>	
備考	プログラムの概要は「4.1WDRに接続」を参照	

3.2.3. ソケットをクローズ

関数名	public static void SocketClose()	
パラメータ	なし	
戻り値	なし	
説明	WDR に接続したソケットをクローズする	
関数の使用方法	<pre>// メイン関数 static void Main() { // WDR に接続 ret = SocketOpen("192.168.10.1", 10002); if (ret == -1) { return; } // ソケットをクローズ SocketClose(); }</pre>	
備考	プログラムの概要は「4.2 ソケットをクローズ」を参照	

3.2.4. 受信したデータをコマンドごとに分割する

関数名	public static bool RecvDatagujde(byte[] recvdData, int recvSize, ushort mode, out int startPos, out int getSize)	
パラメータ	byte[] recvdData	受信したデータ
	int recvSize	受信したデータのサイズ
	ushort mode	取得対象のコマンドモード
	out int startPos	指定したコマンドモードのデータ開始位置
	out int getSize	指定したコマンドモードのデータサイズ
戻り値	bool	データあり:true、データなし:false
説明	受信したデータ内から指定したコマンドモードの応答データがあるか検索し、データの有無と先頭位置、サイズを返す。	
関数の使用方法	<pre>// 受信関数 static int RecvCommand(ushort mode , out byte[] recvData) { // WDR に接続 Int recvSize = sock.Receive(bytes); if (recvSize < 0) { return; -1 } // 応答コマンドのデータを取得する if (true == RecvDatagujde(bytes, recvSize, mode, out pos, out recvSize)) { return -1; } recvData = new byte[recvSize]; Array.Copy(bytes, pos, recvData, 0, recvSize); }</pre>	
備考	プログラムの概要は「4.3 受信したデータをコマンドごとに分割する」を参照	

3.2.5. 要求コマンドを送信し応答コマンドを受信する

関数名	public static int SendCommand(ushort mode, byte[] sendData, out byte[] recvData, int recvTimeout)	
パラメータ	ushort mode	応答コマンドモード
	byte[] sendData	要求コマンドデータ
	out byte[] recvData	応答コマンドデータ
	int recvTimeout	応答コマンド待ち時間
戻り値	int	成功:0、失敗:0 以外
説明	接続した WDR に要求コマンドを送信して、応答データを返す	
関数の使用方法	<pre>// メイン関数 static void Main() { // WDR に接続 ret = SocketOpen("192.168.10.1", 10002); if (ret == -1) { return; } }</pre>	

	<pre>// 送信データを作成 byte[] sendData = new byte[7]; byte[] recvData; sendData[0] = 0x41; sendData[1] = 0x42; sendData[2] = 0x54; sendData[3] = 0x00; sendData[4] = 0x00; sendData[5] = 0x00; sendData[6] = 0x01; // コマンドを送受信 ret = SendCommand(WDR_COMMAND_MODE_TRANSMITTER_STATUS_REQUEST, sendData, out recvData, WDR_TRANSMITTER_STATUS_REQUEST_TIMEOUT); if (ret != 0) { Debug.WriteLine("failed to send data"); return -1; } // ソケットをクローズ SocketClose(); }</pre>
備考	プログラムの概要は「4.4 要求コマンドを送信し応答コマンドを受信する」を参照

3.2.6. 通知コマンドを受信する

関数名	public static int RecvCommand(ushort mode, out byte[] recvData, int recvTimeout)	
パラメータ	ushort mode	通知コマンドモード
	out byte[] recvData	通知コマンドデータ
	int recvTimeout	通知コマンド待ち時間
戻り値	int	成功:0、失敗:0 以外
説明	接続した WDR の通知コマンドを受信してデータを返す	
関数の使用方法	<pre>// メイン関数 static void Main() { // WDR に接続 ret = SocketOpen("192.168.10.1", 10002); if (ret == -1) { return; } // コマンドを受信 ret = RecvCommand(WDR_COMMAND_MODE_STATUS_CHANGE_NOTICE, out recvData, WDR_STATUS_CHANGE_NOTICE_TIMEOUT); if (ret != 0) { Debug.WriteLine("failed to send data"); return -1; } }</pre>	

	<pre> } // ソケットをクローズ SocketClose(); } </pre>
備考	プログラムの概要は「4.5 通知コマンドを受信する」を参照

3.2.7. 送信機状態変化通知を受信する

関数名	public static int WDR_StatusChangeNoticeCommand(out WDR_STATUS_CHANGE_NOTICE_RECV_DATA Data)	
パラメータ	out WDR_STATUS_CHANGE_NOTICE_RECV_DATA Data	受信データ
戻り値	int	成功:0、失敗:0 以外
説明	送信機状態変化通知コマンドを受信してデータを返す	
関数の使用方法	<pre> // メイン関数 static void Main() { // WDR に接続 ret = SocketOpen("192.168.10.1", 10002); if (ret == -1) { return; } // 送信機状態変化通知コマンドを受信 WDR_STATUS_CHANGE_NOTICE_RECV_DATA Data; Data = new WDR_STATUS_CHANGE_NOTICE_RECV_DATA(); WDR_StatusChangeNoticeCommand(out Data); // ソケットをクローズ SocketClose(); } </pre>	
備考	プログラムの概要は「4.6 送信機状態変化通知を受信する」を参照	

3.2.8. カウント値通知を受信する

関数名	public static int WDR_CountNoticeCommand(out WDR_COUNT_NOTICE_RECV_DATA Data)	
パラメータ	out WDR_COUNT_NOTICE_RECV_DATA Data	受信データ
戻り値	int	成功:0、失敗:0 以外
説明	カウント値通知コマンドを受信してデータを返す	
関数の使用方法	<pre> // メイン関数 static void Main() { // WDR に接続 ret = SocketOpen("192.168.10.1", 10002); </pre>	

	<pre> if (ret == -1) { return; } // カウント値通知コマンドを受信 WDR_COUNT_NOTICE_RECV_DATA Data; Data = new WDR_COUNT_NOTICE_RECV_DATA(); WDR_CountNoticeCommand(out Data); // ソケットをクローズ SocketClose(); } </pre>
備考	プログラムの概要は「4.7 カウント値通知を受信する」を参照

3.2.9. 信号灯表示変化通知を受信する

関数名	public static int WDR.SignalLightChangeNoticeCommand(out WDR_SIGNAL_LIGHT_CHANGE_NOTICE_RECV_DATA Data)	
パラメータ	out WDR_SIGNAL_LIGHT_CHANGE_NOTICE_RECV_DATA Data	受信データ
戻り値	int	成功:0、失敗:0 以外
説明	信号灯表示変化通知コマンドを受信してデータを返す	
関数の使用方法	<pre> // メイン関数 static void Main() { // WDR に接続 ret = SocketOpen("192.168.10.1", 10002); if (ret == -1) { return; } // 信号灯表示変化通知コマンドを受信 WDR_SIGNAL_LIGHT_CHANGE_NOTICE_RECV_DATA Data; Data = new WDR_SIGNAL_LIGHT_CHANGE_NOTICE_RECV_DATA(); WDR.SignalLightChangeNoticeCommand(out Data); // ソケットをクローズ SocketClose(); } </pre>	
備考	プログラムの概要は「4.8 信号灯表示変化通知を受信する」を参照	

3.2.10. 送信機状態取得要求を送信し、送信機状態取得応答を受信する

関数名	public static int WDR_TransmitterStatusRequest(ulong IEEEAddress, out WDR_TRANSMITTER_STATUS_REQUEST_RES_DATA Data)	
パラメータ	ulong IEEEAddress	対象の送信機の IEEE アドレス
	out WDR_TRANSMITTER_STATUS_REQUEST_RES_DATA Data	受信データ
戻り値	int	成功:0、失敗:0 以外
説明	送信機状態取得要求コマンドを送信し、送信機状態取得応答コマンドを受信してデータを返す	
関数の使用方法	<pre>// メイン関数 static void Main() { // WDR に接続 ret = SocketOpen("192.168.10.1", 10002); if (ret == -1) { return; } // 送信機状態取得コマンドを送受信 WDR_TRANSMITTER_STATUS_REQUEST_RES_DATA Data; Data = new WDR_TRANSMITTER_STATUS_REQUEST_RES_DATA(); WDR_TransmitterStatusRequest(IEEEAddress, out Data); // ソケットをクローズ SocketClose(); }</pre>	
備考	プログラムの概要は「4.9 送信機状態取得要求を送信し、送信機状態取得応答を受信する」を参照	

3.2.11. 送信機一覧取得要求を送信し、送信機一覧取得応答を受信する

関数名	public static int WDR_TransmitterListRequest(out WDR_TRANSMITTER_LIST_REQUEST_RES_DATA Data)	
パラメータ	out WDR_TRANSMITTER_LIST_REQUEST_RES_DATA Data	受信データ
戻り値	int	成功:0、失敗:0 以外
説明	送信機一覧取得要求コマンドを送信し、送信機一覧取得応答コマンドを受信してデータを返す	
関数の使用方法	<pre>// メイン関数 static void Main() { // WDR に接続 ret = SocketOpen("192.168.10.1", 10002); if (ret == -1) { return; } }</pre>	

	<pre>// 送信機一覧取得コマンドを送受信 WDR_TRANSMITTER_LIST_REQUEST_RES_DATA Data; Data = new WDR_TRANSMITTER_LIST_REQUEST_RES_DATA(); WDR_TransmitterListRequest(out Data); // ソケットをクローズ SocketClose(); }</pre>
備考	プログラムの概要は「4.10 送信機一覧取得要求を送信し、送信機一覧取得応答を受信する」を参照

3.2.12. 送信機情報取得要求を送信し、送信機情報取得応答を受信する

関数名	public static int WDR_TransmitterDataRequest(ulong IEEEAddress, out WDR_TRANSMITTER_DATA_REQUEST_RES_DATA Data, out WDR_TRANSMITTER_DATA_REQUEST_RES_ADD_DATA addData)	
パラメータ	ulong IEEEAddress	対象の送信機の IEEE アドレス
	out WDR_TRANSMITTER_DATA_REQUEST_RES_DATA Data	受信データ
	out WDR_TRANSMITTER_DATA_REQUEST_RES_ADD_DATA addData	拡張受信データ
戻り値	int	成功:0、失敗:0 以外
説明	送信機情報取得要求コマンドを送信し、送信機情報取得応答コマンドを受信してデータを返す	
関数の使用方法	<pre>// メイン関数 static void Main() { // WDR に接続 ret = SocketOpen("192.168.10.1", 10002); if (ret == -1) { return; } // 送信機情報取得コマンドを送受信 WDR_TRANSMITTER_DATA_REQUEST_RES_DATA Data; WDR_TRANSMITTER_DATA_REQUEST_RES_ADD_DATA addData; Data = new WDR_TRANSMITTER_DATA_REQUEST_RES_DATA(); addData = new WDR_TRANSMITTER_DATA_REQUEST_RES_ADD_DATA(); WDR_TransmitterDataRequest(IEEEAddress, out Data, out addData); // ソケットをクローズ SocketClose(); }</pre>	
備考	プログラムの概要は「4.11 送信機情報取得要求を送信し、送信機情報取得応答を受信する」を参照	

3.2.13. 送信機呼び出し表示要求を送信し、送信機呼び出し表示応答を受信する

関数名	public static int WDR_TransmitterCallRequest(ulong IEEEAddress, out WDR_TRANSMITTER_CALL_REQUEST_RES_DATA Data)	
パラメータ	ulong IEEEAddress	対象の送信機の IEEE アドレス
	out WDR_TRANSMITTER_CALL_REQUEST_RES_DATA Data	受信データ
戻り値	int	成功:0、失敗:0 以外
説明	送信機呼び出し表示要求コマンドを送信し、送信機呼び出し表示応答コマンドを受信してデータを返す	
関数の使用方法	<pre>// メイン関数 static void Main() { // WDR に接続 ret = SocketOpen("192.168.10.1", 10002); if (ret == -1) { return; } // 送信機呼び出し表示コマンドを送受信 WDR_TRANSMITTER_CALL_REQUEST_RES_DATA Data; Data = new WDR_TRANSMITTER_CALL_REQUEST_RES_DATA(); WDR_TransmitterCallRequest(IEEEAddress, out Data); // ソケットをクローズ SocketClose(); }</pre>	
備考	プログラムの概要は「4.12 送信機呼び出し表示要求を送信し、送信機呼び出し表示応答を受信する」を参照	

3.2.14. シリアルデータ出力要求を送信し、シリアルデータ出力応答を受信する

関数名	public static int WDR_SerialOutputRequest(WDR_SERIAL_OUTPUT_REQ_DATA outputData, out WDR_SERIAL_OUTPUT_RES_DATA Data)	
パラメータ	WDR_SERIAL_OUTPUT_REQ_DATA outputData	送信データ
	out WDR_SERIAL_OUTPUT_RES_DATA Data	受信データ
戻り値	int	成功:0、失敗:0 以外
説明	シリアルデータ出力要求コマンドを送信し、シリアルデータ出力応答コマンドを受信してデータを返す	
関数の使用方法	<pre>// メイン関数 static void Main() { // WDR に接続 ret = SocketOpen("192.168.10.1", 10002); if (ret == -1) { return; } }</pre>	

	<pre>// シリアルデータ出力コマンドを送受信 WDR_SERIAL_OUTPUT_REQ_DATA sendData; WDR_SERIAL_OUTPUT_RES_DATA Data; sendData = new WDR_SERIAL_OUTPUT_REQ_DATA(); Data = new WDR_SERIAL_OUTPUT_RES_DATA(); sendData.IEEEAddress = 0x6CE4DAFFFE010101; sendData.serialNumber = 0x01; sendData.outputData = {0x01, 0x02, 0x03, 0x04, 0x05}; WDR_SerialOutputRequest(sendData, out Data); // ソケットをクローズ SocketClose(); }</pre>
備考	プログラムの概要は「4.13 シリアルデータ出力要求を送信し、シリアルデータ出力応答を受信する」を参照

3.2.15. 信号灯表示制御要求を送信し、信号灯表示制御応答を受信する

関数名	public static int WDR_SignalLightControlRequest(WDR_SIGNAL_LIGHT_CONTROL_REQ_DATA controlData, out WDR_SIGNAL_LIGHT_CONTROL_RES_DATA Data)	
パラメータ	WDR_SIGNAL_LIGHT_CONTROL_REQ_DATA controlData	送信データ
	out WDR_SIGNAL_LIGHT_CONTROL_RES_DATA Data	受信データ
戻り値	int	成功:0、失敗:0 以外
説明	信号灯表示制御要求コマンドを送信し、信号灯表示制御応答コマンドを受信してデータを返す	
関数の使用方法	<pre>// メイン関数 static void Main() { // WDR に接続 ret = SocketOpen("192.168.10.1", 10002); if (ret == -1) { return; } // 信号灯表示制御コマンドを送受信 WDR_SIGNAL_LIGHT_CONTROL_REQ_DATA sendData; WDR_SIGNAL_LIGHT_CONTROL_RES_DATA Data; sendData = new WDR_SERIAL_OUTPUT_REQ_DATA(); Data = new WDR_SIGNAL_LIGHT_CONTROL_RES_DATA(); sendData.IEEEAddress = 0x6CE4DAFFFE010101; sendData.controlTime= 0x00; sendData.redUnit= 0x11; sendData.yellowUnit= 0x11; sendData.greenUnit= 0x11;</pre>	

	<pre> sendData.blueUnit= 0x11; sendData.whiteUnit= 0x11; sendData.buzzerUnit= 0x11; WDR_SignalLightControlRequest(sendData, out Data); // ソケットをクローズ SocketClose(); } </pre>
備考	プログラムの概要は「4.14 信号灯表示制御要求を送信し、信号灯表示制御応答を受信する」を参照

3.2.16. 信号灯表示解除要求を送信し、信号灯表示解除応答を受信する

関数名	public static int WDR_SignalLightLiftRequest(ulong IEEEAddress, out WDR_SIGNAL_LIGHT_LIFT_RES_DATA Data)	
パラメータ	ulong IEEEAddress	対象の送信機の IEEE アドレス
	out WDR_SIGNAL_LIGHT_LIFT_RES_DATA Data	受信データ
戻り値	int	成功:0、失敗:0 以外
説明	信号灯表示解除要求コマンドを送信し、信号灯表示解除応答コマンドを受信してデータを返す	
関数の使用方法	<pre> // メイン関数 static void Main() { // WDR に接続 ret = SocketOpen("192.168.10.1", 10002); if (ret == -1) { return; } // 信号灯表示解除コマンドを送受信 WDR_SIGNAL_LIGHT_LIFT_RES_DATA Data; Data = new WDR_SIGNAL_LIGHT_LIFT_RES_DATA(); WDR_SignalLightLiftRequest(IEEEAddress, out Data); // ソケットをクローズ SocketClose(); } </pre>	
備考	プログラムの概要は「4.15 信号灯表示解除要求を送信し、信号灯表示解除応答を受信する」を参照	

3.2.17. カウント値登録要求を送信し、カウント値登録応答を受信する

関数名	public static int WDR_SignalLightCountSetRequest(WDR_SIGNAL_LIGHT_COUNT_SET_REQ_DATA setData, out WDR_SIGNAL_LIGHT_COUNT_SET_RES_DATA Data)	
パラメータ	WDR_SIGNAL_LIGHT_COUNT_SET_REQ_DATA setData	送信データ
	out WDR_SIGNAL_LIGHT_COUNT_SET_RES_DATA Data	受信データ
戻り値	int	成功:0、失敗:0 以外
説明	カウント値登録要求コマンドを送信し、カウント値登録応答コマンドを受信してデータを返す	
関数の使用方法	<pre>// メイン関数 static void Main() { // WDR に接続 ret = SocketOpen("192.168.10.1", 10002); if (ret == -1) { return; } // カウント値登録コマンドを送受信 WDR_SIGNAL_LIGHT_COUNT_SET_REQ_DATA sendData; WDR_SIGNAL_LIGHT_COUNT_SET_RES_DATA Data; sendData = new WDR_SIGNAL_LIGHT_COUNT_SET_REQ_DATA(); Data = new WDR_SIGNAL_LIGHT_COUNT_SET_RES_DATA(); sendData.IEEEAddress = 0x6CE4DAFFFE010101; sendData.setCount = 0x01; WDR_SignalLightCountSetRequest(sendData, out Data); // ソケットをクローズ SocketClose(); }</pre>	
備考	プログラムの概要は「4.16 カウント値登録要求を送信し、カウント値登録応答を受信する」を参照	

3.2.18. 受信機情報取得要求を送信し、受信機情報取得応答を受信する

関数名	public static int WDR_ReceiveDataRequest(out WDR_RECEIVER_DATA_REQUEST_RES_DATA Data)	
パラメータ	out WDR_RECEIVER_DATA_REQUEST_RES_DATA Data	受信データ
戻り値	int	成功:0、失敗:0 以外
説明	受信機情報取得要求コマンドを送信し、受信機情報取得応答コマンドを受信してデータを返す	
関数の使用方法	<pre>// メイン関数 static void Main() { // WDR に接続 ret = SocketOpen("192.168.10.1", 10002); if (ret == -1) {</pre>	

	<pre> return; } // 受信機情報取得コマンドを送受信 WDR_RECEIVER_DATA_REQUEST_RES_DATA Data; Data = new WDR_RECEIVER_DATA_REQUEST_RES_DATA(); WDR_ReceiveDataRequest(out Data); // ソケットをクローズ SocketClose(); } </pre>
備考	プログラムの概要は「4.17 受信機情報取得要求を送信し、受信機情報取得応答を受信する」を参照

3.2.19. 受信機リセット要求を送信し、受信機リセット応答を受信する

関数名	public static int WDR_ReceiverResetRequest(out WDR_RECEIVER_RESET_RES_DATA Data)	
パラメータ	out WDR_RECEIVER_RESET_RES_DATA Data	受信データ
戻り値	int	成功:0、失敗:0 以外
説明	受信機リセット要求コマンドを送信し、受信機リセット応答コマンドを受信してデータを返す	
関数の使用方法	<pre> // メイン関数 static void Main() { // WDR に接続 ret = SocketOpen("192.168.10.1", 10002); if (ret == -1) { return; } // 受信機リセットコマンドを送受信 WDR_RECEIVER_RESET_RES_DATA Data; Data = new WDR_RECEIVER_RESET_RES_DATA(); WDR_ReceiverResetRequest(out Data); // ソケットをクローズ SocketClose(); } </pre>	
備考	プログラムの概要は「4.18 受信機リセット要求を送信し、受信機リセット応答を受信する」を参照	

3.2.20. カウント値取得要求を送信し、カウント値取得応答を受信する

関数名	public static int WDR_SignalLightCountGetRequest(ulong IEEEAddress, out WDR_SIGNAL_LIGHT_COUNT_GET_RES_DATA Data)	
パラメータ	ulong IEEEAddress	対象の送信機の IEEE アドレス
	out WDR_SIGNAL_LIGHT_COUNT_GET_RES_DATA Data	受信データ
戻り値	int	成功:0、失敗:0 以外
説明	カウント値取得要求コマンドを送信し、カウント値取得応答コマンドを受信してデータを返す	
関数の使用方法	<pre>// メイン関数 static void Main() { // WDR に接続 ret = SocketOpen("192.168.10.1", 10002); if (ret == -1) { return; } // カウント値取得コマンドを送受信 WDR_SIGNAL_LIGHT_COUNT_GET_RES_DATA Data; Data = new WDR_SIGNAL_LIGHT_COUNT_GET_RES_DATA(); WDR_SignalLightCountGetRequest(IEEEAddress, out Data); // ソケットをクローズ SocketClose(); }</pre>	
備考	プログラムの概要は「4.19 カウント値取得要求を送信し、カウント値取得応答を受信する」を参照	

3.2.21. 信号灯表示取得要求を送信し、信号灯表示取得応答を受信する

関数名	public static int WDR_SignalLightDataGetRequest(ulong IEEEAddress, out WDR_SIGNAL_LIGHT_DATA_GET_RES_DATA Data)	
パラメータ	ulong IEEEAddress	対象の送信機の IEEE アドレス
	out WDR_SIGNAL_LIGHT_DATA_GET_RES_DATA Data	受信データ
戻り値	int	成功:0、失敗:0 以外
説明	信号灯表示取得要求コマンドを送信し、信号灯表示取得応答コマンドを受信してデータを返す	
関数の使用方法	<pre>// メイン関数 static void Main() { // WDR に接続 ret = SocketOpen("192.168.10.1", 10002); if (ret == -1) { return; } // 信号灯表示取得コマンドを送受信 WDR_SIGNAL_LIGHT_DATA_GET_RES_DATA Data;</pre>	

	<pre>Data = new WDR_SIGNAL_LIGHT_DATA_GET_RES_DATA(); WDR_SignalLightDataGetRequest(IEEEAddress, out Data); // ソケットをクローズ SocketClose(); }</pre>
備考	プログラムの概要は「4.20 信号灯表示取得要求を送信し、信号灯表示取得応答を受信する」を参照

3.3. 定数説明

3.3.1. 受信タイムアウトコード

定数名	値	説明
WSAETIMEDOUT	10060	受信タイムアウト判定コード

3.3.2. 製品区分

定数名	値	説明
WDR_PRODUCT_ID	0x5842	WDR の製品区分

3.3.3. WDR 識別子

定数名	値	説明
WDR_COMMAND	0x01	WDR コマンド識別子コード

3.3.4. 拡張

定数名	値	説明
WDR_EXPANSION	0x00	WDR コマンド拡張コード

3.3.5. コマンド種別

定数名	値	説明
WDR_COMMAND_KIND_NOTICE	0x10	通知コマンド
WDR_COMMAND_KIND_REQUEST	0x20	要求コマンド
WDR_COMMAND_KIND_RESPONSE	0x30	応答コマンド

3.3.6. コマンドモード

定数名	値	説明
WDR_COMMAND_MODE_STATUS_CHANGE_NOTICE	0x2001	送信機状態変化通知
WDR_COMMAND_MODE_COUNT_NOTICE	0x2007	カウント値通知
WDR_COMMAND_MODE_SIGNAL_LIGHT_CHANGE_NOTICE	0x2008	信号灯表示変化通知
WDR_COMMAND_MODE_TRANSMITTER_STATUS_REQUEST	0x2002	送信機状態取得
WDR_COMMAND_MODE_TRANSMITTER_LIST_REQUEST	0x2003	送信機一覧取得
WDR_COMMAND_MODE_TRANSMITTER_DATA_REQUEST	0x2004	送信機情報取得
WDR_COMMAND_MODE_TRANSMITTER_CALL_REQUEST	0x4010	送信機呼び出し表示
WDR_COMMAND_MODE_SERIAL_OUTPUT_REQUEST	0x9011	シリアルデータ出力
WDR_COMMAND_MODE_SIGNAL_LIGHT_CONTROL_REQUEST	0xE001	信号灯表示制御要求
WDR_COMMAND_MODE_SIGNAL_LIGHT_CONTROL_RESPONSE	0xE000	信号灯表示制御応答
WDR_COMMAND_MODE_SIGNAL_LIGHT_LIFT_REQUEST	0xE0FF	信号灯表示解除
WDR_COMMAND_MODE_SIGNAL_LIGHT_COUNT_SET_REQUEST	0x6001	カウント値登録

WDR_COMMAND_MODE_RECEIVER_DATA_REQUEST	0x2005	受信機情報取得
WDR_COMMAND_MODE_RECEIVER_RESET_REQUEST	0x2006	受信機リセット
WDR_COMMAND_MODE_SIGNAL_LIGHT_COUNT_GET_REQUEST	0x2009	カウント値取得
WDR_COMMAND_MODE_SIGNAL_LIGHT_DATA_GET_REQUEST	0x200A	信号灯表示取得

3.3.7. コマンドモード別受信タイムアウト時間

定数名	値	説明
WDR_STATUS_CHANGE_NOTICE_TIMEOUT	60*1000	送信機状態変化通知用
WDR_COUNT_NOTICE_TIMEOUT	60*1000	カウント値通知
WDR_SIGNAL_LIGHT_CHANGE_NOTICE_TIMEOUT	60*1000	信号灯表示変化通知
WDR_TRANSMITTER_STATUS_REQUEST_TIMEOUT	2*100	送信機状態取得
WDR_TRANSMITTER_LIST_REQUEST_TIMEOUT	2*1000	送信機一覧取得
WDR_TRANSMITTER_DATA_REQUEST_TIMEOUT	2*1000	送信機情報取得
WDR_TRANSMITTER_CALL_REQUEST_TIMEOUT	15*1000	送信機呼び出し表示
WDR_SERIAL_OUTPUT_REQUEST_TIMEOUT	15*1000	シリアルデータ出力
WDR_SIGNAL_LIGHT_CONTROL_TIMEOUT	15*1000	信号灯表示制御用
WDR_SIGNAL_LIGHT_LIFT_TIMEOUT	15*1000	信号灯表示解除
WDR_SIGNAL_LIGHT_COUNT_SET_TIMEOUT	2*1000	カウント値登録
WDR_RECEIVER_DATA_REQUEST_TIMEOUT	2*1000	受信機情報取得
WDR_RECEIVER_RESET_REQUEST_TIMEOUT	2*1000	受信機リセット
WDR_SIGNAL_LIGHT_COUNT_GET_TIMEOUT	2*1000	カウント値取得
WDR_SIGNAL_LIGHT_DATA_GET_TIMEOUT	2*1000	信号灯表示取得

3.3.8. PNS コマンドの応答データ

定数名	値	説明
PNS_ACK	0x06	正常応答
PNS_NAK	0x15	異常応答

3.4. 構造体説明

3.4.1. バージョン構造体

名前	WDR_VERSION_DATA
定義	<pre>public class WDR_VERSION_DATA { // メジャーバージョン public byte major = 0; // マイナーバージョン public byte minor = 0; };</pre>
説明	応答コマンド内のバージョン番号構造体

3.4.2. WDT バージョン情報構造体

名前	WDR_WDT_VERSION_DATA
定義	<pre>public class WDR_WDT_VERSION_DATA { // メジャーバージョン public byte major = 0; // マイナーバージョン public byte minor = 0; // ダミーデータ public byte dummy = 0; };</pre>
説明	応答コマンド内の WDT バージョン情報構造体

3.4.3. WDT 情報構造体

名前	WDR_INFO_DATA
定義	<pre>public class WDR_INFO_DATA { // バージョン情報 public WDR_WDT_VERSION_DATA version = null; // ステータス情報 public byte status = 0; };</pre>
説明	応答コマンド内の WDT 情報構造体

3.4.4. ベースユニット情報構造体

名前	WDR_BASEUNIT_DATA
定義	<pre>public class WDR_BASEUNIT_DATA { // ユニット形式 public byte format = 0; // バージョン情報 public WDR_WDT_VERSION_DATA version = null; // ディップスイッチ情報</pre>

	<pre> public byte dipSwitch = 0; }; </pre>
説明	応答コマンド内のベースユニット情報構造体

3.4.5. WDT 状態情報構造体

名前	WDR_WDT_STATUS_DATA
定義	<pre> public class WDR_WDT_STATUS_DATA { // WDT の IEEE アドレス public ulong IEEEAddress = 0; // WDT の登録状態 public byte registration = 0; // WDT の接続状態 public byte connect = 0; }; </pre>
説明	応答コマンド内の WDT 状態情報構造体

3.4.6. RS232C データ情報構造体

名前	WDR_RS232C_DATA
定義	<pre> public class WDR_RS232C_DATA { // 入力情報サイズ public byte size = 0; // 通し番号 public byte serialNumber = 0; // 入力情報 public byte[] data = new byte [60]; }; </pre>
説明	応答コマンド内の RS232C データ情報構造体

3.4.7. 送信機状態変化通知構造体

名前	WDR_STATUS_CHANGE_NOTICE_RECV_DATA
定義	<pre> public class WDR_STATUS_CHANGE_NOTICE_RECV_DATA { // IEEE アドレス public ulong IEEEAddress = 0; // 通し番号 public uint serialNumber = 0; // 時刻情報 public ulong time = 0; // バージョン情報 public WDR_VERSION_DATA version = null; // 動作モード public byte actionMode = 0; // WDT 情報 public WDR_INFO_DATA wdtData = null; // ベースユニット情報 }; </pre>

	<pre> public WDR_BASEUNIT_DATA baseUnitData = null; // 信号灯情報(赤) public byte redUnit = 0; // 信号灯情報(黄) public byte yellowUnit = 0; // 信号灯情報(緑) public byte greenUnit = 0; // 信号灯情報(青) public byte blueUnit = 0; // 信号灯情報(白) public byte whiteUnit = 0; // ブザー情報 public byte buzzerUnit = 0; // WDT 監視情報 public byte surveillance = 0; // 外部入力情報 public byte externalInput = 0; // RS232C データ public WDR_RS232C_DATA RS232CData = null; }; </pre>
説明	送信機状態変化通知コマンドの受信データ構造体

3.4.8. カウント値通知構造体

名前	WDR_COUNT_NOTICE_RECV_DATA
定義	<pre> public class WDR_COUNT_NOTICE_RECV_DATA { // IEEE アドレス public ulong IEEEAddress = 0; // 時刻情報 public ulong time = 0; // バージョン情報 public WDR_VERSION_DATA version = null; // 動作モード public byte actionMode = 0; // WDT 情報 public WDR_INFO_DATA wdtData = null; // ベースユニット情報 public WDR_BASEUNIT_DATA baseUnitData = null; // カウント値 public uint countValue = 0; }; </pre>
説明	カウント値通知コマンドの受信データ構造体

3.4.9. 信号灯表示変化通知構造体

名前	WDR_SIGNAL_LIGHT_CHANGE_NOTICE_RECV_DATA
定義	<pre> public class WDR_SIGNAL_LIGHT_CHANGE_NOTICE_RECV_DATA { // IEEE アドレス public ulong IEEEAddress = 0; // 時刻情報 public ulong time = 0; // バージョン情報 public WDR_VERSION_DATA version = null; // 動作モード public byte actionMode = 0; // WDT 情報 public WDR_INFO_DATA wdtData = null; // ベースユニット情報 public WDR_BASEUNIT_DATA baseUnitData = null; // 赤ユニット public byte redUnit = 0; // 黄ユニット public byte yellowUnit = 0; // 緑ユニット public byte greenUnit = 0; // 青ユニット public byte blueUnit = 0; // 白ユニット public byte whiteUnit = 0; // ブザーユニット public byte buzzerUnit = 0; }; </pre>
説明	信号灯表示変化通知コマンドの受信データ構造体

3.4.10. 送信機状態取得構造体

名前	WDR_TRANSMITTER_STATUS_REQUEST_RES_DATA
定義	<pre> public class WDR_TRANSMITTER_STATUS_REQUEST_RES_DATA { // 応答ステータス public byte controlState = 0; // 時刻情報 public ulong time = 0; // バージョン情報 public WDR_VERSION_DATA version = null; // 動作モード public byte actionMode = 0; // WDT 情報 public WDR_INFO_DATA wdtData = null; // ベースユニット情報 public WDR_BASEUNIT_DATA baseUnitData = null; }; </pre>

	<pre>// 信号灯情報(赤) public byte redUnit = 0; // 信号灯情報(黄) public byte yellowUnit = 0; // 信号灯情報(緑) public byte greenUnit = 0; // 信号灯情報(青) public byte blueUnit = 0; // 信号灯情報(白) public byte whiteUnit = 0; // ブザー情報 public byte buzzerUnit = 0; // WDT 監視情報 public byte surveillance = 0; // 外部入力情報 public byte externalInput = 0; // RS232C データ public WDR_RS232C_DATA RS232CData = null; };</pre>
説明	送信機状態取得コマンドの受信データ構造体

3.4.11. 送信機一覧取得構造体

名前	WDR_TRANSMITTER_LIST_REQUEST_RES_DATA
定義	<pre>public class WDR_TRANSMITTER_LIST_REQUEST_RES_DATA { // 応答ステータス public byte controlState = 0; // 取得台数 public byte unitCount = 0; // WDT 状態情報 public WDR_WDT_STATUS_DATA[] wdtStatus = new WDR_WDT_STATUS_DATA[70]; };</pre>
説明	送信機一覧取得コマンドの受信データ構造体

3.4.12. 送信機情報取得構造体

名前	WDR_TRANSMITTER_DATA_REQUEST_RES_DATA
定義	<pre>public class WDR_TRANSMITTER_DATA_REQUEST_RES_DATA { // 応答ステータス public byte controlState = 0; // ユーザネーム public byte[] userName = new byte[121]; // バージョン情報 public WDR_VERSION_DATA version = null; // 動作モード public byte actionMode = 0; // WDT 情報 public WDR_INFO_DATA wdtData = null; };</pre>

	<pre>// ベースユニット情報 public WDR_BASEUNIT_DATA baseUnitData = null; // ExtendedPanID public ulong extendedPanID = 0; // 周波数チャンネル public uint frequencyChannel = 0; // 信号灯入力判定 public byte signalLightInputJudge = 0; // 電源設定 public byte powerSetting = 0; // カウンタ設定 public byte counterSetting = 0; // 送信モード public ushort sendMode = 0; };</pre>
説明	送信機情報取得コマンドの受信データ構造体

3.4.13. 送信機情報取得拡張構造体

名前	WDR_TRANSMITTER_DATA_REQUEST_RES_ADD_DATA
定義	<pre>public class WDR_TRANSMITTER_DATA_REQUEST_RES_ADD_DATA { // 入力情報伝達方式 public byte inputDataTranform = 0; // 信号灯フォーマット public byte signalLightFormat = 0; // 定期送信 public byte regularSend = 0; // 同時入力判定感度設定 public byte concInputSensitiveSetting = 0; // 受信データファイルフォーマット public byte recvDataFileFormat = 0; // 通信設定ボーレート public byte baudrate = 0; // 通信設定データ長 public byte dataLength = 0; // 通信設定パリティ public byte parity = 0; // 通信設定ストップビット public byte stopBit = 0; };</pre>
説明	WDT_6LR_Z2_PRO の時に追加される送信機情報取得コマンドの受信データ構造体

3.4.14. 送信機呼び出し表示構造体

名前	WDR_TRANSMITTER_CALL_REQUEST_RES_DATA
定義	<pre>public class WDR_TRANSMITTER_CALL_REQUEST_RES_DATA { // 応答ステータス public byte controlState = 0; };</pre>
説明	送信機呼び出し表示コマンドの受信データ構造体

3.4.15. シリアルデータ出力要求構造体

名前	WDR_SERIAL_OUTPUT_REQ_DATA
定義	<pre>public class WDR_SERIAL_OUTPUT_REQ_DATA { // IEEE アドレス public ulong IEEEAddress = 0; // 通し番号 public byte serialNumber = 0; // 出力情報 public List<byte> outputData = new List<byte>(); };</pre>
説明	シリアルデータ出力コマンドの送信データ構造体

3.4.16. シリアルデータ出力応答構造体

名前	WDR_SERIAL_OUTPUT_RES_DATA
定義	<pre>public class WDR_SERIAL_OUTPUT_RES_DATA { // 応答ステータス public byte controlState = 0; };</pre>
説明	シリアルデータ出力コマンドの受信データ構造体

3.4.17. 信号灯表示制御要求構造体

名前	WDR_SIGNAL_LIGHT_CONTROL_REQ_DATA
定義	<pre>public class WDR_SIGNAL_LIGHT_CONTROL_REQ_DATA { // IEEE アドレス public ulong IEEEAddress = 0; // 制御時間 public byte controlTime = 0; // 赤ユニット public byte redUnit = 0; // 黄ユニット public byte yellowUnit = 0; // 緑ユニット };</pre>

	<pre> public byte greenUnit = 0; // 青ユニット public byte blueUnit = 0; // 白ユニット public byte whiteUnit = 0; // ブザーユニット public byte buzzerUnit = 0; }; </pre>
説明	信号灯表示制御コマンドの送信データ構造体

3.4.18. 信号灯表示制御応答構造体

名前	WDR_SIGNAL_LIGHT_CONTROL_RES_DATA
定義	<pre> public class WDR_SIGNAL_LIGHT_CONTROL_RES_DATA { // 応答ステータス public byte recvState = 0; // 制御状態 public byte controlState = 0; // 赤ユニット public byte redUnit = 0; // 黄ユニット public byte yellowUnit = 0; // 緑ユニット public byte greenUnit = 0; // 青ユニット public byte blueUnit = 0; // 白ユニット public byte whiteUnit = 0; // ブザーユニット public byte buzzerUnit = 0; }; </pre>
説明	信号灯表示制御コマンドの受信データ構造体

3.4.19. 信号灯表示解除構造体

名前	WDR_SIGNAL_LIGHT_LIFT_RES_DATA
定義	<pre> public class WDR_SIGNAL_LIGHT_LIFT_RES_DATA { // 応答ステータス public byte recvState = 0; // 制御状態 public byte controlState = 0; // 赤ユニット public byte redUnit = 0; // 黄ユニット public byte yellowUnit = 0; // 緑ユニット public byte greenUnit = 0; // 青ユニット }; </pre>

	<pre> public byte blueUnit = 0; // 白ユニット public byte whiteUnit = 0; // ブザーユニット public byte buzzerUnit = 0; }; </pre>
説明	信号灯表示解除コマンドの受信データ構造体

3.4.20. カウント値登録要求構造体

名前	WDR_SIGNAL_LIGHT_COUNT_SET_REQ_DATA
定義	<pre> public class WDR_SIGNAL_LIGHT_COUNT_SET_REQ_DATA { // IEEE アドレス public ulong IEEEAddress = 0; // カウント登録値 public uint setCount = 0; }; </pre>
説明	カウント値登録コマンドの送信データ構造体

3.4.21. カウント値登録応答構造体

名前	WDR_SIGNAL_LIGHT_COUNT_SET_RES_DATA
定義	<pre> public class WDR_SIGNAL_LIGHT_COUNT_SET_RES_DATA { // 応答ステータス public byte controlState = 0; }; </pre>
説明	カウント値登録コマンドの受信データ構造体

3.4.22. 受信機情報取得構造体

名前	WDR_RECEIVER_DATA_REQUEST_RES_DATA
定義	<pre> public class WDR_RECEIVER_DATA_REQUEST_RES_DATA { // 応答ステータス public byte controlState = 0; // ExtendedPanID public ulong extendedPanID = 0; // 周波数チャンネル public uint frequencyChannel = 0; // ファームウェアバージョン public WDR_VERSION_DATA version = null; // ネットワーク状態 public byte networkStatus = 0; // ネットワーク起動方法 public byte networkBoot = 0; // 動作中 ExtendedPanID public ulong actionExtendedPanID = 0; }; </pre>

	<pre>// 動作中周波数チャンネル public byte actionFrequencyChannel = 0; };</pre>
説明	受信機情報コマンドの受信データ構造体

3.4.23. 受信機リセット構造体

名前	WDR_RECEIVER_RESET_RES_DATA
定義	<pre>public class WDR_RECEIVER_RESET_RES_DATA { // 応答ステータス public byte controlState = 0; };</pre>
説明	受信機リセットコマンドの受信データ構造体

3.4.24. カウント値取得構造体

名前	WDR_SIGNAL_LIGHT_COUNT_GET_RES_DATA
定義	<pre>public class WDR_SIGNAL_LIGHT_COUNT_GET_RES_DATA { // 応答ステータス public byte controlState = 0; // 時刻情報 public ulong time = 0; // バージョン情報 public WDR_VERSION_DATA version = null; // 動作モード public byte actionMode = 0; // WDT 情報 public WDR_INFO_DATA wdtData = null; // ベースユニット情報 public WDR_BASEUNIT_DATA baseUnitData = null; // カウント値 public uint count = 0; };</pre>
説明	カウント値取得コマンドの受信データ構造体

3.4.25. 信号灯表示取得構造体

名前	WDR_SIGNAL_LIGHT_DATA_GET_RES_DATA
定義	<pre>public class WDR_SIGNAL_LIGHT_DATA_GET_RES_DATA { // 応答ステータス public byte controlState = 0; // 時刻情報 public ulong time = 0;</pre>

	<pre>// バージョン情報 public WDR_VERSION_DATA version = null; // 動作モード public byte actionMode = 0; // WDT 情報 public WDR_INFO_DATA wdtData = null; // ベースユニット情報 public WDR_BASEUNIT_DATA baseUnitData = null; // 赤ユニット public byte redUnit = 0; // 黄ユニット public byte yellowUnit = 0; // 緑ユニット public byte greenUnit = 0; // 青ユニット public byte blueUnit = 0; // 白ユニット public byte whiteUnit = 0; // ブザーユニット public byte buzzerUnit = 0; };</pre>
説明	信号灯表示取得コマンドの受信データ構造体

4. プログラム概要

プログラムの動作を要点のみ記載する。

4.1. WDR に接続

プログラム	説明
Program.cs <pre>private static Socket sock = null;</pre>	→ソケットのメンバ変数を定義
Program.cs SocketOpen() <pre>public static int SocketOpen(string ip, int port) { try { // Set IP address and port IPAddress ipAddress = IPAddress.Parse(ip); IPEndPoint remoteEP = new IPEndPoint(ipAddress // Creating a socket sock = new Socket(ipAddress.AddressFamily, Soc if (sock == null) { Console.WriteLine("failed to create socket return -1; } // Connect to WDR sock.Connect(remoteEP); } catch (Exception ex) { Console.WriteLine("socket open error"); if (sock != null) { sock.Close(); } return -1; } return 0; }</pre>	→機器の IP アドレスとポート番号を指定 画面から入力した IP アドレス 画面から入力したポート番号 →ソケットのインスタンスを作成 →ソケットの Connect 関数で機器に接続

4.2. ソケットをクローズ

プログラム	説明
<pre>Program.cs SocketClose() public static void SocketClose() { if (sock != null) { //Closing the socket. sock.Shutdown(SocketShutdown.Both); sock.Close(); } }</pre>	→ソケットをシャットダウンしてからクローズを呼び出す

4.3. 受信したデータをコマンドごとに分割する

プログラム	説明
<pre> Program.cs RecvDatagujde() public static bool RecvDatagujde(byte[] recvdata, int recvS [bool ret = false; ushort resSize = 0; ushort resMode = 0; startPos = 0; getSize = 0; // If the reception size is 0 or less, it has not been if (recvSize <= 0) { return ret; } do { // Get the size of one response resSize = (ushort)IPAddress.NetworkToHostOrder(BitC // Get command resMode = (ushort)IPAddress.NetworkToHostOrder(BitC // Command judgment if (resMode == mode) { // If it is the target command, set the size fo getSize = resSize + 6; ret = true; break; } // Add the size of one response to startPos startPos += (resSize + 6); } while (recvSize > startPos); // When startPos become return ret;] </pre>	→受信データからデータサイズを取得 →受信データからコマンドモードを取得する →コマンドモードが指定したモードならデータのサイズをセットして「コマンドあり」を返す。 →コマンドごとのデータの先頭位置を算出 →先頭位置がサイズよりも大きくなるまで繰り返し。

4.4. 要求コマンドを送信し応答コマンドを受信する

プログラム	説明
<pre> Program.cs SendCommand() public static int SendCommand(ushort mode, byte[] sendData) { int ret; recvData = null; try { if (sock == null) { Console.WriteLine("socket is not"); return -1; } // Set timeout sock.SendTimeout = 1000; sock.ReceiveTimeout = recvTimeout; // send ret = sock.Send(sendData); if (ret < 0) { Console.WriteLine("failed to send"); return -1; } // Receive response data byte[] bytes = new byte[1024]; int recvSize = 0; int pos; while (true) { // Data reception recvSize = sock.Receive(bytes); if (recvSize < 0) { Console.WriteLine("failed to recv"); return -1; } // Get the data of the target command from the received data if (true == RecvDataguide(bytes, recvSize, mode, out pos)) { break; } } recvData = new byte[recvSize]; Array.Copy(bytes, pos, recvData, 0, recvSize); } } </pre>	→作成した送信データを Send 関数で送信 →送信後に Recive 関数で機器からのレスポンスを取得 →受信したデータに指定コマンドのデータがあるか判定し、データの先頭位置とサイズを取得する。

```
// Judgment of command error response
ushort commandSize = (ushort)IPAddress.NetworkToHostOrder(Bitl
bool errorFlag = false;
string errorMessage = "";
if (commandSize == 0x0000)
{
    errorFlag = true;
    switch (recvData[17])
    {
        case 0x80:
            errorMessage = "Command error";
            break;

        case 0x81:
            errorMessage = "Mode error";
            break;

        case 0x82:
            errorMessage = "Data error";
            break;

        case 0x83:
            errorMessage = "Connection unit error";
            break;

        case 0x84:
            errorMessage = "Wireless module response error";
            break;

        case 0x86:
            errorMessage = "Data acquisition error";
            break;

        case 0xC0:
            errorMessage = "Initialization abnormality";
            break;

        case 0xFF:
            errorMessage = "Exception anomaly";
            break;

        default:
            errorFlag = false;
            break;
    }
}

if (errorFlag)
{
    Console.WriteLine(errorMessage);
    return -1;
}
```

→コマンドエラー応答の可能性がある
のでサイズを判定

→応答ステータスがコマンドエラー応答
のどれかならメッセージを表示してエラ
ー終了する。

→応答ステータスがコマンドエラー応答
でないなら問題ないので正常終了する

```
}  
catch (SocketException e)  
{  
    if (e.ErrorCode == WSAETIMEDOUT)  
    {  
        Console.WriteLine("TimeOut");  
        return -1;  
    }  
}  
catch (Exception ex)  
{  
    Console.WriteLine(ex.Message);  
    return -1;  
}  
  
return 0;  
}
```

→通信タイムアウトを判定しエラー終了する。

4.5. 通知コマンドを受信する

プログラム	説明
<pre>Program.cs RecvCommand() public static int RecvCommand(ushort mode, out byte[] recvD { recvData = null; try { if (sock == null) { Console.WriteLine("socket is not"); return -1; } // Set timeout sock.ReceiveTimeout = recvTimeout; // Receive notification data byte[] bytes = new byte[1024]; int recvSize = 0; int pos; while (true) { // Data reception recvSize = sock.Receive(bytes); if (recvSize < 0) { Console.WriteLine("failed to recv"); return -1; } // Get the data of the target command from the if (true == RecvDatagujde(bytes, recvSize, mode) { break; } } recvData = new byte[recvSize]; Array.Copy(bytes, pos, recvData, 0, recvSize); } }</pre>	→Recv 関数で機器からの通知を取得 →受信したデータに指定コマンドのデータがあるか判定し、データの先頭位置とサイズを取得する。

```
    }  
    catch (SocketException e)  
    {  
        if (e.ErrorCode == WSAETIMEDOUT)  
        {  
            Console.WriteLine("TimeOut");  
            return -1;  
        }  
    }  
    catch (Exception ex)  
    {  
        Console.WriteLine(ex.Message);  
        return -1;  
    }  
  
    return 0;  
}
```

→通信タイムアウトを判定しエラー終了する。

4.6. 送信機状態変化通知を受信する

プログラム	説明
<pre> Program.cs WDR_StatusChangeNoticeCommand() public static int WDR_StatusChangeNoticeCommand(out WDR_STATUS_CHANGE_NOTICE_RE { int ret; Data = new WDR_STATUS_CHANGE_NOTICE_RECV_DATA(); // Command reception byte[] recvData; ret = RecvCommand(WDR_COMMAND_MODE_STATUS_CHANGE_NOTICE, out recvData, WDR_ // Check for response data if (recvData == null) { // Exception error (including timeout) occurred return -1; } // Check the response data if (recvData[0] == PNS_NAK) { // Receive an abnormal response Console.WriteLine("negative acknowledge"); return -1; } // IEEE address Data.IFFAddress = (ulong)IPAddress.NetworkToHostOrder(BitConverter.ToInt64 // serial number Data.serialNumber = (uint)IPAddress.NetworkToHostOrder(BitConverter.ToInt32 // time information Data.time = (ulong)IPAddress.NetworkToHostOrder(BitConverter.ToInt64(recvDa // version information Data.version = new WDR_VERSION_DATA { major = recvData[30], // Major version minor = recvData[31], // Minor version }; // action mode Data.actionMode = recvData[32]; // WDT information Data.wdtData = new WDR_INFO_DATA { version = new WDR_WDT_VERSION_DATA // version information { major = recvData[33], // Major version minor = recvData[34], // Minor version dummy = recvData[35] // Fixed value }, status = recvData[36], // Status information }; </pre>	→「4.5 通知コマンドを受信する」を呼び出し、通知コマンドを受信 →受信後に異常応答でないか確認 →受信したデータを構造体に格納

```
// Base unit information
Data.baseUnitData = new WDR_BASEUNIT_DATA
{
    format = recvData[37], // Unit type
    version = new WDR_WDT_VERSION_DATA // version information
    {
        major = recvData[38], // Major version
        minor = recvData[39], // Minor version
        dummy = recvData[40] // Fixed value
    },
    dipSwitch = recvData[41] // DIP switch informati
};

// Signal light information (red)
Data.redUnit = recvData[47];

// Signal light information (yellow)
Data.yellowUnit = recvData[48];

// Signal light information (green)
Data.greenUnit = recvData[49];

// Signal light information (blue)
Data.blueUnit = recvData[50];

// Signal light information (white)
Data.whiteUnit = recvData[51];

// Buzzer information
Data.buzzerUnit = recvData[52];

// WDT monitoring information
Data.surveillance = recvData[53];

// External input information
Data.externalInput = recvData[54];

// RS232C data
Data.RS232CData = new WDR_RS232C_DATA();
Data.RS232CData.size = recvData[55];
Data.RS232CData.serialNumber = recvData[56];
Data.RS232CData.data = new byte[60];
Array.Copy(recvData, 57, Data.RS232CData.data, 0, Data.RS232CData.dat

return 0;
}
```

4.7. カウント値通知を受信する

プログラム	説明
<pre> Program.cs WDR_CountNoticeCommand() public static int WDR_CountNoticeCommand(out WDR_COUNT_NOTICE_RECV_DATA { int ret; Data = new WDR_COUNT_NOTICE_RECV_DATA(); // Command reception byte[] recvData; ret = RecvCommand(WDR_COMMAND_MODE_COUNT_NOTICE, out recvData, WDR // Check for response data if (recvData == null) { // Exception error (including timeout) occurred return -1; } // Check for response data if (recvData == null) { // Exception error (including timeout) occurred return -1; } // Check the response data if (recvData[0] == PNS_NAK) { // Receive an abnormal response Console.WriteLine("negative acknowledge"); return -1; } // IEEE address Data.IEEEAddress = (ulong)IPAddress.NetworkToHostOrder(BitConverter // Time information Data.time = (ulong)IPAddress.NetworkToHostOrder(BitConverter.ToInt // version information Data.version = new WDR_VERSION_DATA { major = recvData[30], // Major version minor = recvData[31], // Minor version }; // action mode Data.actionMode = recvData[32]; // WDT information Data.wdtData = new WDR_INFO_DATA { version = new WDR_WDT_VERSION_DATA // version information { major = recvData[33], // Major version minor = recvData[34], // Minor version dummy = recvData[35] // Fixed value }, status = recvData[36], // Status information }; </pre>	→「4.5 通知コマンドを受信する」を呼び出し、通知コマンドを受信 →受信後に異常応答でないか確認 →受信したデータを構造体に格納


```
// Base unit information
Data.baseUnitData = new WDR_BASEUNIT_DATA
{
    format = recvData[37],           // Unit type
    version = new WDR_WDT_VERSION_DATA // version informat
    {
        major = recvData[38],       // Major version
        minor = recvData[39],       // Minor version
        dummy = recvData[40]        // Fixed value
    },
    dipSwitch = recvData[41]        // DIP switch infor
};

// Count value
Data.countValue = (uint)IPAddress.NetworkToHostOrder(BitConvert

return 0;
}
```

4.8. 信号灯表示変化通知を受信する

プログラム	説明
<pre> Program.cs WDR_SignalLightChangeNoticeCommand() public static int WDR_SignalLightChangeNoticeCommand(out WDR_SIGNAL_LIGHT_CHANGE_NOTICE_RECV_DATA out r) { int ret; Data = new WDR_SIGNAL_LIGHT_CHANGE_NOTICE_RECV_DATA(); // Command reception byte[] recvData; ret = RecvCommand(WDR_COMMAND_MODE_SIGNAL_LIGHT_CHANGE_NOTICE, out r); // Check for response data if (recvData == null) { // Exception error (including timeout) occurred return -1; } // Check the response data if (recvData[0] == PNS_NAK) { // Receive an abnormal response Console.WriteLine("negative acknowledge"); return -1; } // IEEE address Data.IEEEAddress = (ulong)IPAddress.NetworkToHostOrder(BitConverter.ToInt64(recvData[1])); // Time information Data.time = (ulong)IPAddress.NetworkToHostOrder(BitConverter.ToInt64(recvData[2])); // version information Data.version = new WDR_VERSION_DATA { major = recvData[30], // Major version minor = recvData[31], // Minor version }; // action mode Data.actionMode = recvData[32]; // WDT information Data.wdtData = new WDR_INFO_DATA { version = new WDR_WDT_VERSION_DATA // version information { major = recvData[33], // Major version minor = recvData[34], // Minor version dummy = recvData[35] // Fixed value }, status = recvData[36], // Status information }; } </pre>	→「4.5 通知コマンドを受信する」を呼び出し、通知コマンドを受信 →受信後に異常応答でないか確認 →受信したデータを構造体に格納

```
// Base unit information
Data.baseUnitData = new WDR_BASEUNIT_DATA
{
    format = recvData[37],           // Unit type
    version = new WDR_WDT_VERSION_DATA // version info
    {
        major = recvData[38],       // Major versio
        minor = recvData[39],       // Minor versio
        dummy = recvData[40]        // Fixed value
    },
    dipSwitch = recvData[41]        // DIP switch i
};

// Red unit
Data.redUnit = recvData[47];

// Yellow unit
Data.yellowUnit = recvData[48];

// Green unit
Data.greenUnit = recvData[49];

// Blue unit
Data.blueUnit = recvData[50];

// White unit
Data.whiteUnit = recvData[51];

// Buzzer unit
Data.buzzerUnit = recvData[52];

return 0;
}
```

4.9. 送信機状態取得要求を送信し、送信機状態取得応答を受信する

プログラム	説明
<pre> Program.cs WDR_TransmitterStatusRequest() public static int WDR_TransmitterStatusRequest(ulong IEEEAddress, { int ret; Data = new WDR_TRANSMITTER_STATUS_REQUEST_RES_DATA(); try { byte[] sendData = []; // Product category sendData = sendData.Concat(BitConverter.GetBytes(WDR_PRODUC // identifier sendData = sendData.Concat(new byte[] { WDR_COMMAND }).To# // Expansion sendData = sendData.Concat(new byte[] { WDR_EXPANSION }).1 // size sendData = sendData.Concat(BitConverter.GetBytes((ushort)C // Command type sendData = sendData.Concat(new byte[] { WDR_COMMAND_KIND_F // IEEE address sendData = sendData.Concat(BitConverter.GetBytes((ulong)IE // Command mode sendData = sendData.Concat(BitConverter.GetBytes(WDR_COMM# // Send request command byte[] rcvData; ret = SendCommand(WDR_COMMAND_MODE_TRANSMITTER_STATUS_REQUES if (ret != 0) { Console.WriteLine("failed to send data"); return -1; } // Check for response data if (rcvData == null) { // Exception error (including timeout) occurred return -1; } // Check the response data if (rcvData[0] == PNS_NAK) { // Receive an abnormal response Console.WriteLine("negative acknowledge"); return -1; } } } </pre>	→送信データを作成 →「4.4 要求コマンドを送信し応答コマンドを受信する」を呼び出し、コマンドを送受信 →受信後に異常応答でないか確認

```
// Response status
Data.controlState = recvData[1];

// Time information
Data.time = (ulong)IPAddress.NetworkToHostOrder(BitConverter.ToInt64(recvData[2]));

// version information
Data.version = new WDR_VERSION_DATA
{
    major = recvData[30],    // Major version
    minor = recvData[31],    // Minor version
};

// action mode
Data.actionMode = recvData[32];

// WDT information
Data.wdtData = new WDR_INFO_DATA
{
    version = new WDR_WDT_VERSION_DATA // version information
    {
        major = recvData[33],    // Major version
        minor = recvData[34],    // Minor version
        dummy = recvData[35]    // Fixed value
    },
    status = recvData[36],    // Status information
};

// Base unit information
Data.baseUnitData = new WDR_BASEUNIT_DATA
{
    format = recvData[37],    // Unit type
    version = new WDR_WDT_VERSION_DATA // version informat
    {
        major = recvData[38],    // Major version
        minor = recvData[39],    // Minor version
        dummy = recvData[40]    // Fixed value
    },
    dipSwitch = recvData[41]    // DIP switch infor
};

// Signal light information (red)
Data.redUnit = recvData[47];

// Signal light information (yellow)
Data.yellowUnit = recvData[48];

// Signal light information (green)
Data.greenUnit = recvData[49];

// Signal light information (blue)
Data.blueUnit = recvData[50];

// Signal light information (white)
Data.whiteUnit = recvData[51];
```

→受信したデータを構造体に格納

```
// Buzzer information
Data.buzzerUnit = recvData[52];

// WDT monitoring information
Data.surveillance = recvData[53];

// External input information
Data.externalInput = recvData[54];

// RS232C data
Data.RS232CData = new WDR_RS232C_DATA();
Data.RS232CData.size = recvData[55];
Data.RS232CData.serialNumber = recvData[56];
Data.RS232CData.data = new byte[60];
Array.Copy(recvData, 57, Data.RS232CData.data, 0, Data.RS2

}
catch (Exception ex)
{
    Console.WriteLine(ex.Message);
    return -1;
}

return 0;
}
```

4.10.送信機一覧取得要求を送信し、送信機一覧取得応答を受信する

[illegible]

```
// Check the response data
if (recvData[0] == PNS_NAK)
{
    // Receive an abnormal response
    Console.WriteLine("negative acknowledge");
    return -1;
}

// Response status
Data.controlState = recvData[17];

// Number of acquisitions
Data.unitCount = recvData[18];

// WDT status information
for (int count = 0; count < Data.unitCount; count++)
{
    // Object creation
    WDR_WDT_STATUS_DATA setData = new WDR_WDT_STATUS_DATA()

    // IEEE address
    setData.IEEEAddress = (ulong)IPAddress.NetworkToHostOrder(recvData[20 + (count * 10)]);

    // Registration status
    setData.registration = recvData[27 + (count * 10)];

    // Connection Status
    setData.connect = recvData[28 + (count * 10)];

    // Set in an array
    Data.wdtStatus[count] = setData;
}
}
catch (Exception ex)
{
    Console.WriteLine(ex.Message);
    return -1;
}

return 0;
}
```

→受信後に異常応答でないか
確認

→受信したデータを構造体に
格納

4.11.送信機情報取得要求を送信し、送信機情報取得応答を受信する

プログラム	説明
<pre> Program.cs WDR_TransmitterDataRequest() public static int WDR_TransmitterDataRequest(ulong IEEEAddress, out WDR_TRANS { int ret; Data = new WDR_TRANSMITTER_DATA_REQUEST_RES_DATA(); addData = null; try { byte[] sendData = { }; // Product category sendData = sendData.Concat(BitConverter.GetBytes(WDR_PRODUCT_ID).Rev // identifier sendData = sendData.Concat(new byte[] { WDR_COMMAND }).ToArray(); // Expansion sendData = sendData.Concat(new byte[] { WDR_EXPANSION }).ToArray(); // size sendData = sendData.Concat(BitConverter.GetBytes((ushort)0x000B).Rev // Command type sendData = sendData.Concat(new byte[] { WDR_COMMAND_KIND_REQUEST }); // IEEE address sendData = sendData.Concat(BitConverter.GetBytes((ulong)IEEEAddress // Command mode sendData = sendData.Concat(BitConverter.GetBytes(WDR_COMMAND_MODE_TF // Send request command byte[] recvData; ret = SendCommand(WDR_COMMAND_MODE_TRANSMITTER_DATA_REQUEST, if (ret != 0) { Console.WriteLine("failed to send data"); return -1; } // Check for response data if (recvData == null) { // Exception error (including timeout) occurred return -1; } // Check the response data if (recvData[0] == PNS_NAK) { // Receive an abnormal response Console.WriteLine("negative acknowledge"); return -1; } } } </pre>	→送信データを作成 →「4.4 要求コマンドを送信し応答コマンドを受信する」を呼び出し、コマンドを送受信 →受信後に異常応答でないか確認

```
// Response status
Data.controlState = recvData[17];

// User name
Data.userName = new byte[121];
Array.Copy(recvData, 18, Data.userName, 0, Data.userName.Length);

// version information
Data.version = new WDR_VERSION_DATA
{
    major = recvData[139],    // Major version
    minor = recvData[140],   // Minor version
};

// action mode
Data.actionMode = recvData[141];

// WDT information
Data.wdtData = new WDR_INFO_DATA
{
    version = new WDR_WDT_VERSION_DATA // version information
    {
        major = recvData[142],    // Major version
        minor = recvData[143],    // Minor version
        dummy = recvData[144]     // Fixed value
    },
    status = recvData[145],       // Status information
};

// Base unit information
Data.baseUnitData = new WDR_BASEUNIT_DATA
{
    format = recvData[146],        // Unit type
    version = new WDR_WDT_VERSION_DATA // version informati
    {
        major = recvData[147],    // Major version
        minor = recvData[148],    // Minor version
        dummy = recvData[149]     // Fixed value
    },
    dipSwitch = recvData[150]     // DIP switch inform

};

// ExtendedPanID
Data.extendedPanID = (ulong)IPAddress.NetworkToHostOrder(BitConv

// Frequency channel
Data.frequencyChannel = (uint)IPAddress.NetworkToHostOrder(BitCc

// Signal light input judgment
Data.signalLightInputJudge = recvData[168];

// Power settings
Data.powerSetting = recvData[169];

// Counter setting
Data.counterSetting = recvData[170];

// Send mode
Data.sendMode = (ushort)IPAddress.NetworkToHostOrder(BitConverte
```

→受信したデータを構造体に格納

```
// In the case of WDT-6LR-Z2-PRO, the expansion structure is also
if (Data.actionMode == 0xFF)
{
    addData = new WDR_TRANSMITTER_DATA_REQUEST_RES_ADD_DATA();

    // Input information transmission method
    addData.inputDataTranform = recvData[173];

    // Signal light format
    addData.signalLightFormat = recvData[174];

    // Periodic transmission
    addData.regularSend = recvData[175];

    // Simultaneous input judgment sensitivity setting
    addData.concInputSensitiveSetting = recvData[176];

    // Received data file format
    addData.recvDataFileFormat = recvData[177];

    // Communication setting baud rate
    addData.baudrate = recvData[178];

    // Communication setting data length
    addData.dataLength = recvData[179];

    // Communication setting parity
    addData.parity = recvData[180];

    // Communication setting stop bit
    addData.stopBit = recvData[181];
}

}

catch (Exception ex)
{
    Console.WriteLine(ex.Message);
    return -1;
}

return 0;
}
```

4.12. 送信機呼び出し表示要求を送信し、送信機呼び出し表示応答を受信する

プログラム	説明
<pre> Program.cs WDR_TransmitterCallRequest () public static int WDR_TransmitterCallRequest(ulong IEEEAddress, out WDR { int ret; Data = new WDR_TRANSMITTER_CALL_REQUEST_RES_DATA(); try { byte[] sendData = { }; // Product category sendData = sendData.Concat(BitConverter.GetBytes(WDR_PRODUCT_ID // identifier sendData = sendData.Concat(new byte[] { WDR_COMMAND }).ToArray(// Expansion sendData = sendData.Concat(new byte[] { WDR_EXPANSION }).ToArra // size sendData = sendData.Concat(BitConverter.GetBytes((ushort)0x000B // Command type sendData = sendData.Concat(new byte[] { WDR_COMMAND_KIND_REQUES // IEEE address sendData = sendData.Concat(BitConverter.GetBytes((ulong)IEEEAdd // Command mode sendData = sendData.Concat(BitConverter.GetBytes(WDR_COMMAND_MO // Send request command byte[] recvData; ret = SendCommand(WDR_COMMAND_MODE_TRANSMITTER_CALL_RE if (ret != 0) { Console.WriteLine("failed to send data"); return -1; } // Check for response data if (recvData == null) { // Exception error (including timeout) occurred return -1; } // Check the response data if (recvData[0] == PNS_NAK) { // Receive an abnormal response Console.WriteLine("negative acknowledge"); return -1; } } } </pre>	→送信データを作成 →「4.4 要求コマンドを送信し応答コマンドを受信する」を呼び出し、コマンドを送受信 →受信後に異常応答でないか確認

<pre>// Response status Data.controlState = recvData[17]; } catch (Exception ex) { Console.WriteLine(ex.Message); return -1; } return 0; }</pre>	→受信したデータを構造体に格納
--	-----------------

4.13. シリアルデータ出力要求を送信し、シリアルデータ出力応答を受信する

[illegible]

<pre>// Check for response data if (recvData == null) { // Exception error (including timeout) occurred return -1; } // Check the response data if (recvData[0] == PNS_NAK) { // Receive an abnormal response Console.WriteLine("negative acknowledge"); return -1; } // Response status Data.controlState = recvData[17]; } catch (Exception ex) { Console.WriteLine(ex.Message); return -1; } return 0; }</pre>	→受信後に異常応答でないか確認 →受信したデータを構造体に格納
--	---


```
// Check for response data
if (recvData == null)
{
    // Exception error (including timeout) occurred
    return -1;
}

// Check the response data
if (recvData[0] == PNS_NAK)
{
    // Receive an abnormal response
    Console.WriteLine("negative acknowledge");
    return -1;
}

// Response status
Data.recvState = recvData[17];

// Control state
Data.controlState = recvData[18];

// Red unit
Data.redUnit = recvData[19];

// Yellow unit
Data.yellowUnit = recvData[20];

// Green unit
Data.greenUnit = recvData[21];

// Blue unit
Data.blueUnit = recvData[22];

// White unit
Data.whiteUnit = recvData[23];

// Buzzer unit
Data.buzzerUnit = recvData[24];
}
catch (Exception ex)
{
    Console.WriteLine(ex.Message);
    return -1;
}

return 0;
}
```

→受信後に異常応答でない
か確認

→受信したデータを構造体に
格納


```
// Check for response data
if (recvData == null)
{
    // Exception error (including timeout) occurred
    return -1;
}

// Check the response data
if (recvData[0] == PNS_NAK)
{
    // Receive an abnormal response
    Console.WriteLine("negative acknowledge");
    return -1;
}

// Response status
Data.recvState = recvData[17];

// Control state
Data.controlState = recvData[18];

// Red unit
Data.redUnit = recvData[19];

// Yellow unit
Data.yellowUnit = recvData[20];

// Green unit
Data.greenUnit = recvData[21];

// Blue unit
Data.blueUnit = recvData[22];

// White unit
Data.whiteUnit = recvData[23];

// Buzzer unit
Data.buzzerUnit = recvData[24];
}
catch (Exception ex)
{
    Console.WriteLine(ex.Message);
    return -1;
}

return 0;
}
```

→受信後に異常応答でない
か確認

→受信したデータを構造体に
格納

4.16. カウント値登録要求を送信し、カウント値登録応答を受信する

プログラム	説明
<pre>Program.cs WDR_SignalLightCountSetRequest() public static int WDR_SignalLightCountSetRequest(WDR_SIGNAL_LIGHT_COUNT_SET_RES_DATA resData) { int ret; Data = new WDR_SIGNAL_LIGHT_COUNT_SET_RES_DATA(); try { byte[] sendData = { }; // Product category sendData = sendData.Concat(BitConverter.GetBytes(resData.ProductCategory)); // identifier sendData = sendData.Concat(new byte[] { WDR_COMMAND_MODE_SIGNAL_LIGHT_COUNT_SET_REQUEST }); // Expansion sendData = sendData.Concat(new byte[] { WDR_EXPANSION_CODE_PRODUCT_CATEGORY }); // size sendData = sendData.Concat(BitConverter.GetBytes(sendData.Length)); // Command type sendData = sendData.Concat(new byte[] { WDR_COMMAND_TYPE_PRODUCT_INFORMATION }); // IEEE address sendData = sendData.Concat(BitConverter.GetBytes(IEEEAddress)); // Command mode sendData = sendData.Concat(BitConverter.GetBytes((byte)WDR_COMMAND_MODE_PRODUCT_INFORMATION)); // Count registration value sendData = sendData.Concat(BitConverter.GetBytes(countRegistrationValue)); // Send request command byte[] recvData; ret = SendCommand(WDR_COMMAND_MODE_SIGNAL_LIGHT_COUNT_SET_REQUEST, sendData); if (ret != 0) { Console.WriteLine("failed to send data"); return -1; } // Check for response data if (recvData == null) { // Exception error (including timeout) occurred return -1; } } }</pre>	→送信データを作成 →「4.4 要求コマンドを送信し応答コマンドを受信する」を呼び出し、コマンドを送受信

<pre>// Check the response data if (recvData[0] == PNS_NAK) { // Receive an abnormal response Console.WriteLine("negative acknowledge"); return -1; } // Response status Data.controlState = recvData[17]; } catch (Exception ex) { Console.WriteLine(ex.Message); return -1; } return 0; }</pre>	→受信後に異常応答でないか確認 →受信したデータを構造体に格納
---	---

4.17. 受信機情報取得要求を送信し、受信機情報取得応答を受信する

プログラム	説明
<pre> Program.cs WDR_ReceiveDataRequest() public static int WDR_ReceiveDataRequest(out WDR_RECV_DATA outData) { int ret; Data = new WDR_RECEIVER_DATA_REQUEST_RES_DATA() { ProductCategory = "00000000", Identifier = "00000000", Expansion = "00000000", Size = "00000000", CommandType = "00000000", IEEEAddress = "00000000", CommandMode = "00000000", SendRequestCommand = "00000000", RecvResponseCommand = "00000000" }; try { byte[] sendData = { }; // Product category sendData = sendData.Concat(BitConverter.GetBytes(ProductCategory)).ToArray(); // identifier sendData = sendData.Concat(new byte[] { WDR_COMMAND_MODE_RECEIVER }).ToArray(); // Expansion sendData = sendData.Concat(new byte[] { WDR_COMMAND_MODE_RECEIVER }).ToArray(); // size sendData = sendData.Concat(BitConverter.GetBytes(Size)).ToArray(); // Command type sendData = sendData.Concat(new byte[] { WDR_COMMAND_MODE_RECEIVER }).ToArray(); // IEEE address byte[] IEEEdata = { 0x00, 0x00, 0x00, 0x00 }; sendData = sendData.Concat(IEEEdata).ToArray(); // Command mode sendData = sendData.Concat(BitConverter.GetBytes(CommandMode)).ToArray(); // Send request command byte[] recvData; ret = SendCommand(WDR_COMMAND_MODE_RECEIVER, sendData); if (ret != 0) { Console.WriteLine("failed to send data"); return -1; } // Check for response data if (recvData == null) { // Exception error (including timeout) return -1; } } catch { } outData = Data; } </pre>	→送信データを作成 →「4.4 要求コマンドを送信し応答コマンドを受信する」を呼び出し、コマンドを送受信

```
// Check the response data
if (recvData[0] == PNS_NAK)
{
    // Receive an abnormal response
    Console.WriteLine("negative acknowledgement");
    return -1;
}

// Response status
Data.controlState = recvData[17];

// ExtendedPanID
Data.extendedPanID = (ulong)IPAddress.NetworkToHostOrder(recvData[18]);

// Frequency channel
Data.frequencyChannel = (uint)IPAddress.NetworkToHostOrder(recvData[19]);

// Firmware version
Data.version = new WDR_VERSION_DATA
{
    major = recvData[30], // Major version
    minor = recvData[31], // Minor version
};

// Network status
Data.networkStatus = recvData[32];

// How to boot the network
Data.networkBoot = recvData[33];

// Running ExtendedPanID
Data.actionExtendedPanID = (ulong)IPAddress.NetworkToHostOrder(recvData[34]);

// Operating frequency channel
Data.actionFrequencyChannel = recvData[42];
}
catch (Exception ex)
{
    Console.WriteLine(ex.Message);
    return -1;
}

return 0;
}
```

→受信後に異常応答でないか確認

→受信したデータを構造体に格納

4.18. 受信機リセット要求を送信し、受信機リセット応答を受信する

プログラム	説明
<pre>Program.cs WDR_ReceiverResetRequest() public static int WDR_ReceiverResetRequest(out WDR_RE { int ret; Data = new WDR_RECEIVER_RESET_RES_DATA(); try { byte[] sendData = []; // Product category sendData = sendData.Concat(BitConverter.GetBy // identifier sendData = sendData.Concat(new byte[] { WDR_C // Expansion sendData = sendData.Concat(new byte[] { WDR_E // size sendData = sendData.Concat(BitConverter.GetBy // Command type sendData = sendData.Concat(new byte[] { WDR_C // IEEE address byte[] IEEEEdata = [0x00, 0x00, 0x00, 0x00, sendData = sendData.Concat(IEEEEdata).ToArray // Command mode sendData = sendData.Concat(BitConverter.GetBy // Send request command byte[] recvData; ret = SendCommand(WDR_COMMAND_MODE_RECEIVER_F if (ret != 0) { Console.WriteLine("failed to send data"); return -1; } // Check for response data if (recvData == null) { // Exception error (including timeout) oc return -1; } } }</pre>	→送信データを作成 →「4.4 要求コマンドを送信し応答コマンドを受信する」を呼び出し、コマンドを送受信


```
// Check the response data
if (recvData[0] == PNS_NAK)
{
    // Receive an abnormal response
    Console.WriteLine("negative acknowledge");
    return -1;
}

// Response status
Data.controlState = recvData[17];
}
catch (Exception ex)
{
    Console.WriteLine(ex.Message);
    return -1;
}

return 0;
}
```

→受信後に異常応答でないか確認

→受信したデータを構造体に格納


```
// Check the response data
if (recvData[0] == PNS_NAK)
{
    // Receive an abnormal response
    Console.WriteLine("negative acknowledged.");
    return -1;
}

// Response status
Data.controlState = recvData[17];

// Time information
Data.time = (ulong)IPAddress.NetworkToHost(

// version information
Data.version = new WDR_VERSION_DATA
{
    major = recvData[30],    // Major v
    minor = recvData[31],    // Minor v
};

// action mode
Data.actionMode = recvData[32];

// WDT information
Data.wdtData = new WDR_INFO_DATA
{
    version = new WDR_WDT_VERSION_DATA {
        major = recvData[33],    // Major v
        minor = recvData[34],    // Minor v
        dummy = recvData[35]    // Fixed v
    },
    status = recvData[36],    // Status
};

// Base unit information
Data.baseUnitData = new WDR_BASEUNIT_DATA
{
    format = recvData[37],
    version = new WDR_WDT_VERSION_DATA
    {
        major = recvData[38],
        minor = recvData[39],
        dummy = recvData[40]
    },
    dipSwitch = recvData[41]
};
```

→受信後に異常応答でないか確認

→受信したデータを構造体に格納

```
        // Count value
        Data.count = (uint)IPAddress.NetworkToHost
    }
    catch (Exception ex)
    {
        Console.WriteLine(ex.Message);
        return -1;
    }

    return 0;
}
```

4.20. 信号灯表示取得要求を送信し、信号灯表示取得応答を受信する

プログラム	説明
<pre>Program.cs WDR_SignalLightDataGetRequest() public static int WDR_SignalLightDataGetRequest(ulong { int ret; Data = new WDR_SIGNAL_LIGHT_DATA_GET_RES_DATA(); try { byte[] sendData = []; // Product category sendData = sendData.Concat(BitConverter.GetBytes(// identifier sendData = sendData.Concat(new byte[] { WDR_COI // Expansion sendData = sendData.Concat(new byte[] { WDR_EXI // size sendData = sendData.Concat(BitConverter.GetBytes(// Command type sendData = sendData.Concat(new byte[] { WDR_COI // IEEE address sendData = sendData.Concat(BitConverter.GetBytes(// Command mode sendData = sendData.Concat(BitConverter.GetBytes(// Send request command byte[] recvData; ret = SendCommand(WDR_COMMAND_MODE_SIGNAL_LIGH if (ret != 0) { Console.WriteLine("failed to send data"); return -1; } // Check for response data if (recvData == null) { // Exception error (including timeout) occur return -1; } }</pre>	→送信データを作成 →「4.4 要求コマンドを送信し応答コマンドを受信する」を呼び出し、 コマンドを送受信

```
// Check the response data
if (recvData[0] == PNS_NAK)
{
    // Receive an abnormal response
    Console.WriteLine("negative acknowledge");
    return -1;
}

// Response status
Data.controlState = recvData[17];

// Time information
Data.time = (ulong)IPAddress.NetworkToHostOrder

// version information
Data.version = new WDR_VERSION_DATA
{
    major = recvData[30],    // Major version
    minor = recvData[31],   // Minor version
};

// action mode
Data.actionMode = recvData[32];

// WDT information
Data.wdtData = new WDR_INFO_DATA
{
    version = new WDR_WDT_VERSION_DATA //
    {
        major = recvData[33],    // Major \
        minor = recvData[34],    // Minor \
        dummy = recvData[35]     // Fixed \
    },
    status = recvData[36],       // Status
};

// Base unit information
Data.baseUnitData = new WDR_BASEUNIT_DATA
{
    format = recvData[37],
    version = new WDR_WDT_VERSION_DATA
    {
        major = recvData[38],
        minor = recvData[39],
        dummy = recvData[40]
    },
    dipSwitch = recvData[41]
};
```

→受信後に異常応答でないか確認

→受信したデータを構造体に格納

```
// Red unit
Data.redUnit = recvData[47];

// Yellow unit
Data.yellowUnit = recvData[48];

// Green unit
Data.greenUnit = recvData[49];

// Blue unit
Data.blueUnit = recvData[50];

// White unit
Data.whiteUnit = recvData[51];

// Buzzer unit
Data.buzzerUnit = recvData[52];
}
catch (Exception ex)
{
    Console.WriteLine(ex.Message);
    return -1;
}

return 0;
}
```